

Assignment 3 | Bridge Pattern

Course: ShP-2216 Software Design Patterns, Astana IT University, 2026-2027 (instructor: Yerassyl Bekenov)

Student: Adilzhan Kuandykov · **Group:** SE-2526 · **Topic:** C — Reports (Report × Formatter)

Repository: <https://github.com/adilzhander/design-pattern-bridge>

Base commit (I1/I2, T1-T5): 0da31fcda61f99ef99560021967baacf06fd89cf

Extension commit (I3, T6-T7): e6ce14442ba2f9fa063fdf7d6062b117bf397f8a

Submitted commit: the head of branch main; its exact hash is given in the Moodle submission text (a file cannot contain the hash of the commit that adds it).

1. Two independent dimensions and why Bridge fits

A school report changes along two directions that do not depend on each other: **what** is reported (attendance rate, grade average — the calculation) and **how** it is presented (plain text, HTML, Markdown — the output format). With inheritance only, every pair would need its own subclass (AttendanceTextReport, AttendanceHtmlReport, GradeTextReport, ...): $2 \times 3 = 6$ classes now, and every new report or format multiplies that number. Bridge separates the dimensions into two hierarchies — the abstraction Report and the implementor Formatter — and joins them by object composition: one interface-typed field Report.formatter, supplied through the constructor and replaceable with setImplementation(Formatter). Each side now grows by adding one class, and the format of an existing report object can be changed at runtime.

Role	Class	Responsibility
Abstraction	bridge.report.Report (abstract)	holds id and the bridge field; execute() builds the report through the formatter
A1 / A2	AttendanceReport / GradeReport	3 of 4 sessions → 75% / average of 70, 80, 90 → 80
Implementor	bridge.format.Formatter (interface)	low-level operations heading, field, document
I1 / I2 / I3	TextFormatter / HtmlFormatter / MarkdownFormatter	plain text / escaped HTML / Markdown output
Client	Main (default package)	creates the combinations and runs checks T1-T7

2. How a call crosses the bridge

Main calls report.execute(). Report.execute() asks the subclass for content() — for AttendanceReport the ordered pairs *attended* → 3/4 and *rate* → 75%; passes each pair to formatter.field(label, value); formats the title with formatter.heading(title() + " " + id); and joins everything with formatter.document(heading, fields). Report never names a concrete formatter: there is no new, cast or instanceof on the abstraction side, so any class that implements Formatter works. In T5, Main keeps one AttendanceReport object (original), switches it with current.setImplementation(new HtmlFormatter()) and checks current == original. Only the formatter field is re-pointed; id, attended, totalSessions and the 75% rate stay the same, so the second execute() returns HTML with identical data.

3. Bridge vs Adapter

Both place an interface between a client and an implementation and forward calls through a stored reference, but their intent differs. **Bridge** is designed up front: I split Report and Formatter deliberately so that the two hierarchies evolve independently and combine freely (6 combinations, no combination subclasses). **Adapter** is applied afterwards to make an *existing* class with an incompatible interface usable through an interface the client already expects; it converts one interface to another and does not create a second hierarchy. In this project, if a third-party PdfWriter with writeTitle(...) and writeParagraph(...) had to be reused, a PdfFormatterAdapter implements Formatter wrapping it would be an Adapter — it would plug into my bridge as one more implementor, but its purpose would be interface conversion, not separating two dimensions.

4. UML class diagram

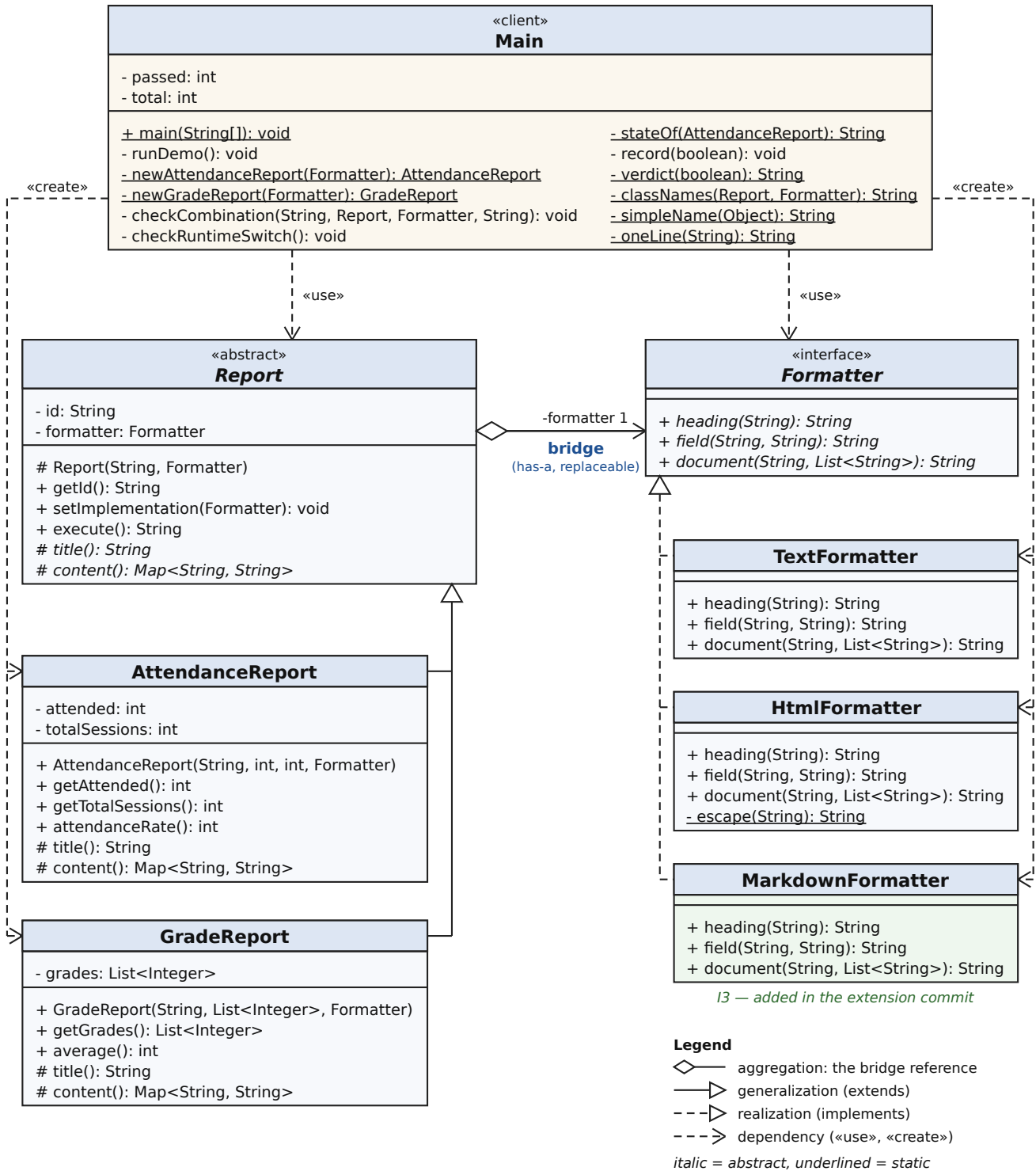


Figure 1. Final code (after the I3 commit). Report stores the interface-typed reference - formatter: Formatter (the bridge); Main creates the concrete classes and uses both abstractions.

5. Clean Code in five excerpts

5.1 Separation of responsibilities — `AttendanceReport.content()` and `HtmlFormatter.field()`

```
protected Map<String, String> content() {
    Map<String, String> content = new LinkedHashMap<>();
    content.put("attended", attended + "/" + totalSessions);
    content.put("rate", attendanceRate() + "%");
    return content;
}

public String field(String label, String value) {
    return "<li>" + escape(label) + ": " + escape(value) + "</li>";
}
```

Benefit: the report only calculates *what* to show and the formatter only decides *how* it looks. A new format never touches the percentage logic, and a new report never touches HTML escaping — each class has one reason to change.

5.2 Meaningful names — the T5 check in `Main.checkRuntimeSwitch()`

```
Report current = original;
current.setImplementation(second);
String after = current.execute();

boolean sameObject = current == original;
boolean stateUnchanged = stateBefore.equals(EXPECTED_STATE) && stateOf(original).equals(stateBefore);
```

Benefit: the code reads like its specification (“same object”, “state unchanged”), so it needs no explanatory comments. The same applies to `attendanceRate()`, `setImplementation(Formatter)` and `newAttendanceReport(...)`.

5.3 Small, focused methods — `HtmlFormatter.escape()` and `Main.verdict()`

```
private static String escape(String text) {
    return text.replace("&", "&amp;")
               .replace("<", "&lt;")
               .replace(">", "&gt;");
}

private static String verdict(boolean pass) {
    return pass ? "PASS" : "FAIL";
}
```

Benefit: each method does one job and can be understood and tested on its own; the escaping rule exists in exactly one place and is reused by `heading()` and `field()`.

5.4 No duplicated workflow — `Report.execute()` and `Main.runDemo()`

```
public String execute() {
    List<String> fields = new ArrayList<>();
    for (Map.Entry<String, String> entry : content().entrySet()) {
        fields.add(formatter.field(entry.getKey(), entry.getValue()));
    }
    return formatter.document(formatter.heading(title() + " " + id), fields);
}

checkCombination("T1", newAttendanceReport(text), text, EXPECTED_T1);
checkCombination("T2", newAttendanceReport(html), html, EXPECTED_T2);
```

Benefit: the “build a report” algorithm is written once for 2 reports × 3 formatters; subclasses only supply `title()` and `content()`. One `checkCombination` method runs T1–T4, T6 and T7, so the PASS/FAIL logic and output format cannot drift apart between checks.

5.5 Encapsulation of state — GradeReport and Report

```
private final List<Integer> grades;

public GradeReport(String id, List<Integer> grades, Formatter formatter) {
    super(id, formatter);
    if (grades.isEmpty()) {
        throw new IllegalArgumentException("grades must not be empty");
    }
    this.grades = List.copyOf(grades);
}
```

Benefit: domain data is private, final and validated; `List.copyOf` stores an unmodifiable copy, so no caller can change the grades later. In `Report`, `id` is final and the private `formatter` can only change through `setImplementation`, which rejects null. This is exactly what T5 relies on when it proves that the data survives a formatter switch.

6. Adding I3 without changing the existing hierarchies

After the base commit `0da31fc` (T1–T5 passing) I added `MarkdownFormatter` implements `Formatter` in a separate commit `e6ce144`. `extension.diff` (`git diff 0da31fc HEAD -- src`) contains exactly two files: the new `MarkdownFormatter.java` (+21 lines) and `Main.java` (+8 lines: one import, two expected strings, one formatter instance and the T6/T7 calls). `Report`, `AttendanceReport`, `GradeReport`, `Formatter`, `TextFormatter` and `HtmlFormatter` are unchanged, because the abstraction depends only on the `Formatter` contract. Outside `src/` only `sources.txt` gained one line. T6 and T7 passed on the first run.

Trade-off. The `Formatter` interface is a fixed contract. Adding a new *format* costs one class, but adding a new kind of *formatting operation* — for example a `table(...)` for one row per student — means changing `Formatter` and every implementor (I1, I2 and I3) at once. Bridge also adds indirection: to understand one output you read two classes instead of one. For a single fixed format this would be over-engineering; here it pays off because both dimensions are expected to grow.

7. Evidence (demo-output.txt)

Check	Participating classes	Status	Actual result (line breaks shown as \n)
T1	AttendanceReport + TextFormatter	PASS	ATTENDANCE REPORT AR-1\nattended: 3/4\nrate: 75%
T2	AttendanceReport + HtmlFormatter	PASS	<article> Attendance Report AR-1 - attended: 3/4 - rate: 75% </article>
T3	GradeReport + TextFormatter	PASS	GRADE REPORT GR-1\ngrades: 70, 80, 90\naverage: 80
T4	GradeReport + HtmlFormatter	PASS	<article> Grade Report GR-1 - grades: 70, 80, 90 - average: 80 </article>
T5	AttendanceReport: TextFormatter → HtmlFormatter	PASS	sameObject=true stateUnchanged=true state=id=AR-1 attended=3/4 rate=75%; before = T1 result, after = T2 result
T6	AttendanceReport + MarkdownFormatter	PASS	# Attendance Report AR-1\n\n- **attended:** 3/4\n- **rate:** 75%
T7	GradeReport + MarkdownFormatter	PASS	# Grade Report GR-1\n\n- **grades:** 70, 80, 90\n- **average:** 80

Final line of the run: SUMMARY: 7/7 PASS. Build and run: `javac --release 17 -encoding UTF-8 -d out "@sources.txt"`, then `java -cp out Main --demo`.

References

- [1] E. Gamma, R. Helm, R. Johnson, J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994 — Adapter (pp. 139–150), Bridge (pp. 151–161).
- [2] A. Shvets. Refactoring.Guru — “Bridge” and “Adapter”. <https://refactoring.guru/design-patterns/bridge>, <https://refactoring.guru/design-patterns/adapter>
- [3] R. C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.
- [4] Oracle. Java SE 17 API Specification (`java.util.List.copyOf`, `java.util.Objects.requireNonNull`). <https://docs.oracle.com/en/java/javase/17/docs/api/>
- [5] Y. Bekenov. Assignment 3 | Bridge Pattern — instructions, ShP-2216, Astana IT University, 2026.